



Fundamentals of Software Engineering

LECTURE 08: WRITING REQUIREMENTS

How to write GOOD REQUIREMENTS

Constraints of a Good requirement

- Unambiguous
- Testable (verifiable)
- Clear (concise, terse, simple, precise)
- Correct
- Understandable
- Feasible (realistic, possible)
- Independent
- Atomic
- Necessary
- Implementation free (abstract)

Defines the system under discussion

Verb with correct identifier (shall or may)

The Online Banking System shall allow the Internet user to access her current account balance in less than 5 seconds.

Defines a positive end result

Quality criteria

- Identifies the system under discussion and a desired end result that is wanted within a specified time that is measurable
- The challenge is to seek out the system under discussion, end result, and success measure in every requirement

Writing a good requirement

Unambiguous

- There should be only **one way to interpret the requirement**. Sometimes ambiguity is introduced by undefined acronyms:
 - ***REQ1The system shall be implemented using ASP.***
 - Does ASP mean Active Server Pages or Application Service Provider? To fix this, we can mention a full name and provide an acronym in parentheses:
 - ***REQ1The system shall be implemented using Active Server Pages (ASP).***
 - Here's another example:
 - ***REQ1The system shall not accept passwords longer than 15 characters.***
 - It is not clear what the system is supposed to do:
 - **The system shall not let the user enter more than 15 characters.**
 - **The system shall truncate the entered string to 15 characters.**
 - **The system shall display an error message if the user enters more than 15 characters.**

Testable (Verifiable)

- *The system shall resist concurrent usage by many users.*
- What number should be considered “many”—10, 100, 1,000?
- Some words can make a requirement untestable
- Some adjectives: robust, safe, accurate, effective, efficient, expandable, flexible, maintainable, reliable, user-friendly, adequate
- Some adverbs and adverbial phrases: quickly, safely, in a timely manner
- Nonspecific words or acronyms: etc., and/or, TBD

Clear (Concise, Terse, Simple, Precise)

- Requirements should **not contain unnecessary verbiage** or **information**. They should be stated clearly and simply:
- *REQ1 Sometimes the user will enter Airport Code, which the system will understand, but sometimes the closest city may replace it, so the user does not need to know what the airport code is, and it will still be understood by the system.*
- This sentence may be replaced by a simpler one:
- *REQ1 The system shall identify the airport based on either an Airport Code or a City Name.*

Correct

- If a requirement contains **facts**, these facts should be **true**:
- *REQ1Car rental prices shall show all applicable taxes (including 6% state tax).*
- The tax depends on the state, so the provided 6% figure is incorrect.

Understandable

- Requirements should be grammatically correct and written in a consistent style. Standard conventions should be used. The word “shall” should be used instead of “will,” “must,” or “may.”



Feasible (Realistic, Possible)

- The requirement should be doable within existing constraints such as time, money, and available resources:
- REQ1The system shall have a natural language interface that will understand commands given in English language.*
- This requirement may be not feasible within a short span of development time.

Independent

- To understand the requirement, there should **not be a need to know any other requirement**:
- REQ1The list of available flights shall include flight numbers, departure time, and arrival time for every leg of a flight.*
- REQ2It should be sorted by price.*
- The word “It” in the second sentence refers to the previous requirement. However, if the order of the requirements changes, this requirement will not be understandable.

Atomic

- The requirement should contain a **single traceable element**:
- REQ1The system shall provide the opportunity to book the flight, purchase a ticket, reserve a hotel room, reserve a car, and provide information about attractions.*
- This requirement combines five atomic requirements, which makes traceability very difficult. Sentences including the words **“and” or “but”** should be reviewed to see if they can be broken into atomic requirements.

Necessary

- A requirement is unnecessary if None of the stakeholders needs the requirement. Or Removing the requirement will not affect the system.
- An example of a requirement that is not needed by a stakeholder is a requirement that is added by developers and designers
- For example, the fact that a developer thinks that users would like a feature that displays a map of the airport and he knows how to implement it is not a valid reason to add this requirement.
- An example of a requirement that can be removed because it does not provide any new information might look like the following:
- REQ1All requirements specified in the Vision document shall be implemented and tested.*

Implementation free (Abstract)

- Requirements should not contain unnecessary design and implementation information:
- *REQ1 Content information shall be stored in a text file.*
- How the information is stored is transparent to the user and should be the designer's or architect's decision.

Consistent

- There should **not be any conflicts** between the requirements. Conflicts may be direct or indirect. Direct conflicts occur when, in the same situation, different behavior is expected:
- *REQ1 Dates shall be displayed in the mm/dd/yyyy format.*
- *REQ2 Dates shall be displayed in the dd/mm/yyyy format.*
- *REQ1 For users in the U.S., dates shall be displayed in the mm/dd/yyyy format.*
- *REQ2 For users in France, dates shall be displayed in the dd/mm/yyyy format.*

Nonredundant

- Each requirement should be expressed **only once** and should **not overlap** with another requirement:
- *REQ1A calendar shall be available to help with entering the flight date.*
- *REQ2The system shall display a pop-up calendar when entering any date.*
- The first requirement (related to only the flight date) is a subset of the second one (related to any date entered by the user).

Complete

- A requirement should be specified for all conditions that can occur:
- REQ1 A destination country does not need to be displayed for flights within the U.S.
- REQ2 For overseas flights, the system shall display a destination country.
- What about flights to Canada and Mexico? They are neither “within the U.S.” nor “overseas.”
- All applicable requirements should be specified. This is the toughest condition to be checked. There is really no way to be sure that all the requirements are captured and that one week before the production date one of the stakeholders won’t say, “I forgot to mention that I need one more feature in the application.”

Conflicts

- Conflicts **between different nonfunctional requirements** are common in
 - complex systems, Spacecraft system
 - To minimize weight, the number of separate chips in the system should be minimized
 - To minimize power consumption, lower power chips should be used
 - However, using low power chips may mean that more chips have to be used. Which is the most critical requirement?

Requirement Document

The software requirements document

The software requirements document is the official statement of what is required of the system developers

Can include both a definition of user requirements and a specification of the system requirements

It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it

Requirements and design

- ✧ In principle, requirements should state **what the system should do** and the design should describe **how** it does this.
- ✧ In practice, requirements and design are inseparable
 - A system architecture may be designed to structure the requirements;
 - The system may inter-operate with other systems that generate design requirements;
 - The use of a specific architecture to satisfy non-functional requirements may be a domain requirement.
 - This may be the consequence of a regulatory requirement.

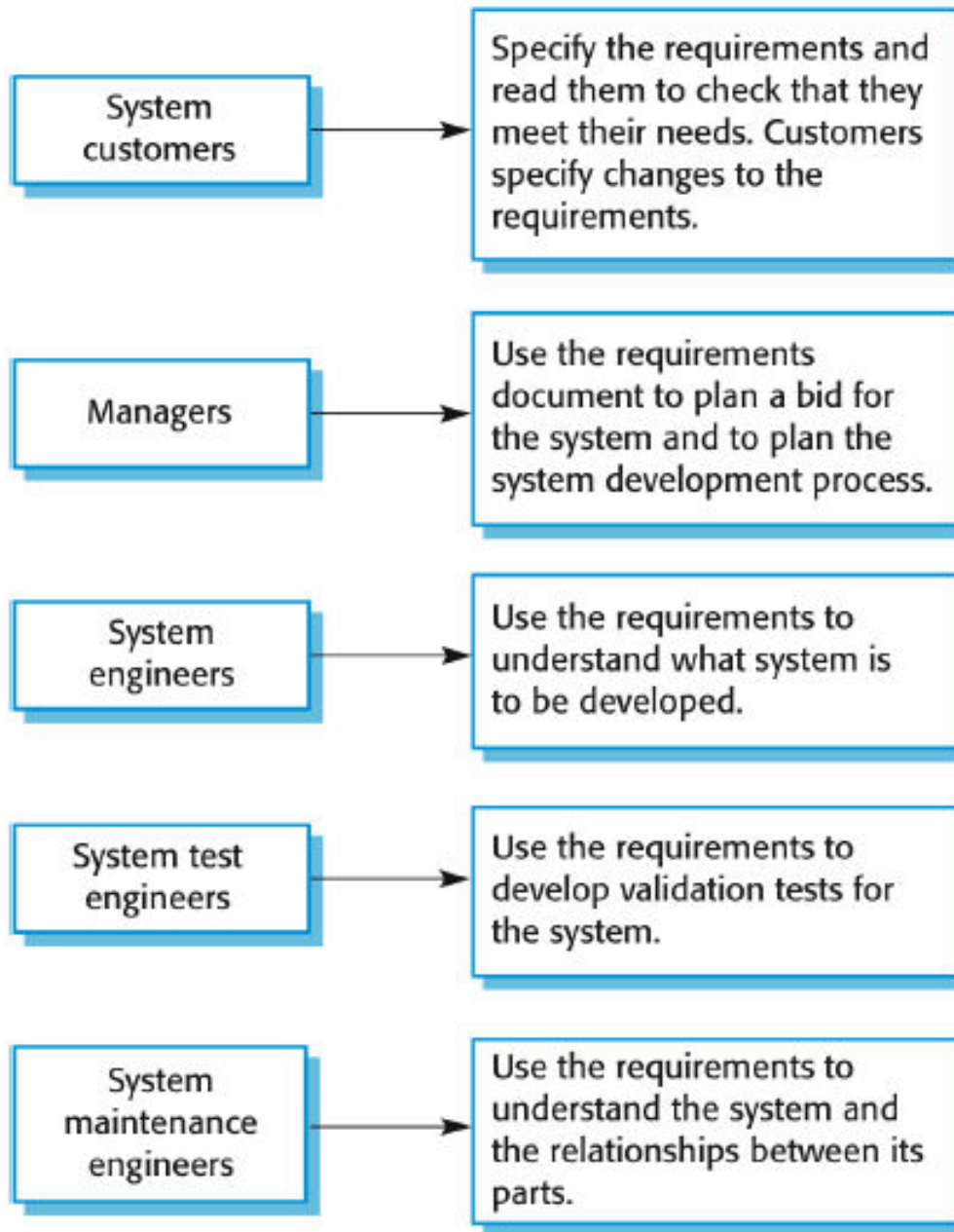
Agile methods and requirements

Many agile methods argue that producing a **requirements document is a waste of time** as requirements change so quickly

The document is therefore **always out of date**

Methods such as **XP use incremental requirements engineering** and **express requirements as 'user stories'** (discussed in Chapter)

This is practical for business systems but problematic for systems that require a lot of **pre delivery analysis** (e.g. critical systems) or **systems developed by several teams**



Users of a requirements document

The structure of a requirements document

Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.

The structure of a requirements document

Chapter	Description
System requirements specification	This should describe the functional and nonfunctional requirements in more detail. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System models	This might include graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information that is related to the application being developed; for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

Requirement's specification

- ❑ The process of writing down the **user and system requirements** in a requirements document
- ❑ **User requirements** have to be **understandable by end users** and **customers** who do not have a technical background
- ❑ **System requirements** are **more detailed requirements** and may include **more technical information**
- ❑ The requirements may be **part of a contract** for the system development
- ❑ It is therefore important that these are as complete as possible

Ways of writing a system requirements specification

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

Natural language specification

- ✧ Requirements are written as **natural language sentences** supplemented by **diagrams and tables**
- ✧ Used for writing requirements because it is expressive, intuitive and universal. This means that the requirements can be understood by users and customers.

Guidelines for writing requirements

- ✧ Create a standard format and use it for all requirements
- ✧ Use language in a consistent way. Use **shall** for mandatory requirements, **should** for desirable requirements.
- ✧ Use text highlighting to identify key parts of the requirement
- ✧ Avoid the use of computer jargon
- ✧ Include an explanation (rationale) of why a requirement is necessary

Problems with natural language

✧Lack of clarity

- Precision is difficult without making the document difficult to read.

✧Requirements confusion

- Functional and non-functional requirements tend to be mixed-up.

✧Requirements amalgamation

- Several different requirements may be expressed together.

Example requirements for the insulin pump software system

3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. *(Changes in blood sugar are relatively slow so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)*

3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. *(A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.)*

Tabular specification

- ✧ Used to supplement natural language
- ✧ Particularly useful when you have to define a number of possible alternative courses of action
- ✧ For example, the insulin pump systems bases its computations on the rate of change of blood sugar level and the tabular specification explains how to calculate the insulin requirement for different scenarios

Tabular specification of computation for an insulin pump

Condition	Action
Sugar level falling ($r_2 < r_1$)	CompDose = 0
Sugar level stable ($r_2 = r_1$)	CompDose = 0
Sugar level increasing and rate of increase decreasing ($(r_2 - r_1) < (r_1 - r_0)$)	CompDose = 0
Sugar level increasing and rate of increase stable or increasing ($(r_2 - r_1) \geq (r_1 - r_0)$)	CompDose = round $((r_2 - r_1)/4)$ If rounded result = 0 then CompDose = MinimumDose

Structured specifications

✧ An approach to writing requirements where the freedom of the requirements writer is limited and requirements are written in a standard way

✧ This works well for some types of requirements, e.g. requirements for embedded control system but is sometimes too rigid for writing business system requirements

A structured specification of a requirement for an insulin pump

Insulin Pump/Control Software/SRS/3.3.2

Function Compute insulin dose: safe sugar level.

Description

Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.

Inputs Current sugar reading (r2); the previous two readings (r0 and r1).

Source Current sugar reading from sensor. Other readings from memory.

Outputs CompDose—the dose in insulin to be delivered.

Destination Main control loop.

A structured specification of a requirement for an insulin pump

Action

CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.

Requirements

Two previous readings so that the rate of change of sugar level can be computed.

Pre-condition

The insulin reservoir contains at least the maximum allowed single dose of insulin.

Post-condition r_0 is replaced by r_1 then r_1 is replaced by r_2 .

Side effects None.

Form based specifications

- ✧ Definition of the **function** or entity
- ✧ Description of **inputs** and where they come from
- ✧ Description of **outputs** and where they go to
- ✧ **Information** about the information needed for the computation and other entities used
- ✧ Description of the **action** to be taken
- ✧ **Pre and post conditions** (if appropriate)
- ✧ The **side effects** (if any) of the function



Exercise

Identify Function and Non-Function Requirements

- The software automatically validates customers against the ABC Contact Management System
- FR
- Employees never allowed to update their salary information. Such attempt should be reported to the security administrator.
- NFR

- Users must change the initially assigned login password immediately after the first successful login. Moreover, the initial should never be reused.
-

- NFR
- The software system should be integrated with banking API
- FR
- Every unsuccessful attempt by a user to access an item of data shall be recorded on an audit trail.
- NFR

-
- The software should be portable. So moving from one OS to other OS does not create any problem.
 - NFR
 - The software system should pass [Section 508](#) accessibility requirement.
 - NFR

- The background color for all windows in the application will be blue and have a hexadecimal RGB color value of 0x0000FF.
-

- FR
- A website should be capable enough to handle 20 million users without affecting its performance
- NFR
- The Sales system should allow users to record customers sales
- FR

Parameters	Functional Requirement	Non-Functional Requirement
What it is	Verb	Attributes
End-result	Product feature	Product properties
Capturing	Easy to capture	Hard to capture
Objective	Helps you verify the functionality of the software.	Helps you to verify the performance of the software.
Area of focus	Focus on user requirement	Concentrates on the user's expectation.
Documentation	Describe what the product does	Describes how the product works
Type of Testing	Functional Testing like System, Integration, End to End, API testing, etc.	Non-Functional Testing like Performance, Stress, Usability, Security testing, etc.
Test Execution	Test Execution is done before non-functional testing.	After the functional testing
Product Info	Product Features	Product Properties

